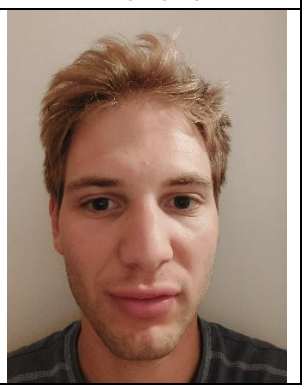
Lab3 – Extending Your Metronome Thing with MQTT

Group name: Boar

Members			
Alessandro Querzé	Andrei Radu	Enrico Giannini	Mirko Tonelli
			

Link to YouTube video

Project Description

We left the previous metronome business logic unchanged, choosing to use a different communication protocol. To achieve this, we transitioned our Raspberry Pi device client from a RESTful request/response model to an event-driven MQTT publish/subscribe architecture.

The local hardware still provides the same operations as before (calculating bpm and changing the learn/play mode), while also publishing these state changes to an MQTT broker, encoding them in JSON. Our topic design structure, that starts with the primary identifier ***metronome/boar***, is as follows (the first 4 are from the remote to the device, while the rest are from the device to whoever is monitoring its state):

- ***cmd/set_bpm***: Used to set a new bpm value to the device
- ***cmd/set_mode***: Used to switch the current mode (*PLAY* or *LEARN*) to the device
- ***cmd/reset_min***: It resets the minimum bpm value reached by the device
- ***cmd/reset_max***: It resets the maximum bpm value reached by the device
- ***state/bpm***: It publishes the current bpm value
- ***state/mode***: It publishes the current mode (*PLAY* or *LEARN*)
- ***state/min***: It publishes the current minimum bpm value
- ***state/max***: It publishes the current maximum bpm value
- ***event/beat***: For every beat of the metronome device, this topic is published with the current bpm value and a sequence number that is incremented at every publication
- ***event/tap***: The device publishes the number of times the button determining the bpms in learn mode was pressed
- ***status***: States if the device is *ONLINE* or *OFFLINE*

Of these topics, we chose the *state* and *status* ones as **retained**, to ensure that a remote monitor immediately receives the last known state of a device already running, rather than waiting for the next update. We applied a *Quality of Service* of 1 (ensuring that the messages are received) to all topics

except the *event* ones, because they are neither commands that change the state of the device nor state information that the remote monitor want to know once it connects to a device.

Lastly, the dashboard saw little changes, just the ones to adapt to the MQTT communication model.

Distribution of Effort

For this project there wasn't a particular division of task between us, everyone interacted, initially to fix the issues that were found in the Lab2 submission, then to find out how to organize the framework and finally to develop the source files that shape the actual project.